

第 3 次作业 Sprint 1

软件模块设计与风险分析报告

项目：设计检查工具——使用启发式评价方法对 Screen2Vec 描述的 UI 设计进行检查

姓名	王俊淇
角色定位	技术方案负责人
提交内容	Sprint 1 个人承诺、软件模块设计报告、风险分析
作业说明	本文档依据第 3 次作业要求撰写，结合项目章程与第 2 次作业中的 Product Backlog。

摘要：本报告聚焦 Sprint 1 的技术主链路，围绕“规则库准备 - 分析流程设计 - 结构化结果输出”展开任务拆解与模块设计，目标是在 1 周内完成可演示、可解释、可继续迭代的 MVP 核心能力。

一、项目背景与个人角色

根据课程给出的 Sprint 1 作业要求，本次个人提交不仅要说明自己在迭代中的承诺任务，还需要输出软件模块设计报告，并对整个项目进行风险识别与优先级排序。结合项目启动材料与新一轮 Product Backlog，本项目当前仍以“方案调研 + 可演示 MVP”作为主线，首版重点证明：系统能够接收 Screen2Vec 生成的 UI 描述文本，依据启发式评价规则进行分析，并输出结构化的问题与建议。

我在团队中的角色是“技术方案负责人”。因此，本次 Sprint 1 个人工作不追求完成全部系统，而是聚焦技术主链路：一是完善启发式规则库的组织方式；二是设计并实现检查流程的核心逻辑；三是将检查结果转化为可展示、可解释、可汇报的结构化输出。这样的分工与项目章程中的职责保持一致，也与第二次作业中我为项目所定义的 MVP 边界相衔接。

二、Sprint 1 迭代目标与本人承诺

Sprint 1 的整体目标是：在 1 周内完成项目最小可行版本中的核心链路验证，使团队在评审会议上能够展示“输入 - 分析 - 输出”的工作流程，并对后续 Sprint 的扩展方向形成共识。

本人在 Sprint 1 中承诺的用户故事与设计任务如下：

编号	对应用户故事	本人承诺内容	预计工作量	依赖关系	验收方式
US-03	维护启发式规则库	整理 Nielsen 启发式规则，建立“规则名称 - 检查点 - 触发条件 - 输出模板”映射表。	8h	需求边界已明确	形成规则库表与示例输入
US-04	触发开始检查	设计检查执行流程，定义输入校验、规则遍历、结果聚合与异常提示逻辑。	6h	依赖规则库初版	能对示例文本完成一次完整分析
US-06	查看规则依据	在输出结果中保留“命中规则、判定原因、证据片段”字段，提高可解释性。	4h	依赖分析流程	结果可追溯到规则依据
US-07	查看风险说明和优化建议	补充建议生成模板，使每条问题都对应风险说明与可执	6h	依赖规则映射与分类结	输出内容完整、便于汇

		行改进建议。		构	报
--	--	--------	--	---	---

从任务选择上看，我主要承诺 US-03、US-04、US-06、US-07 四个高优先级故事。这些故事连接起规则准备、执行流程、结果解释和建议输出，属于 MVP 的主价值路径。相较于界面美化、筛选导出等体验增强项，这些内容更能直接证明项目技术方案是否成立。

三、Sprint 1 任务拆解、分工与依赖

为了使用户故事可以真正落地执行，我将其进一步翻译为可分配、可检查的软件设计任务，并明确输入、输出、前后依赖与协作对象。具体拆解如下。

序号	任务名称	负责人	计划时间	前置依赖	交付物
1	整理启发式规则库并划分检查维度	王俊淇	Day1	项目目标与需求已明确	规则库初表
2	设计输入结构与字段规范	王俊淇	Day1	任务 1	输入模板/示例
3	设计检查流程与伪代码	王俊淇	Day2	任务 1-2	流程图、伪代码
4	实现规则遍历与结果聚合逻辑	王俊淇	Day3-4	任务 3	核心代码
5	补充风险说明与建议输出模板	王俊淇	Day4	任务 4	输出模板
6	与 PM 对齐汇报结构和演示案例	王俊淇、刘颜硕	Day5	任务 1-5	汇报页素材
7	参加评审并根据反馈修订 Backlog	全体成员	Day6-7	演示完成	改进项清单

依赖关系上，规则库整理是整个技术方案的起点；只有当规则项及其检查维度明确后，检查执行流程和输出结构才能稳定。与团队协作方面，我需要和 PM 对齐 Sprint 目标与展示口径，也需要和负责背景分析、规则竞品的同学保持信息同步，确保最终汇报不会出现“方案表达”和“技术实现”脱节的问题。

四、软件模块设计报告

4.1 设计方法

本项目采用“问题驱动 + MVP 优先 + 结构化输出”的设计方法。首先，从课程项目的真实约束出发，将复杂的 UI 可用性评审问题缩减为可在课堂场景中验证的文本分析问题；其次，坚持 MVP 思路，只保留最关键的输入、分析、输出闭环；最后，强调结构化结果输出，使系统结论不仅“能给出”，还必须“能解释、能引用、能汇报”。

4.2 开发环境

项目项	选择方案	说明
开发语言	Python 3.11	便于快速实现规则逻辑、文本处理与示例验证
代码生成辅助	大语言模型 + 自动代码生成工具	用于加速流程设计、样例代码与文档整理
规则存储	JSON / Python 字典	适合首版快速迭代和结构化维护
版本管理	Git / GitHub	课程要求中需体现代码管理过程；当前仓库由小组统一创建
文档规范	Markdown + Word/PDF	便于技术记录与课程提交

4.3 系统总体思路

系统整体采用轻量级流水线结构：输入层接收 Screen2Vec 生成的 UI 描述文本；分析层依据启发式规则进行逐项判断；输出层对发现的问题进行分类汇总，并给出风险说明与改进建议。这样设计的优点是链路清晰、易于演示，也方便后续增加更多规则或接入更强的模型。

输入 UI 描述	规则解析与匹配	问题分类与证据生成	结构化结果输出
----------	---------	-----------	---------

4.4 技术原理

- (1) Screen2Vec 描述文本作为输入基础。与直接识别 UI 截图相比，文本描述能够显著降低视觉解析成本，也更符合本课程项目“方案调研型”的边界。
- (2) 启发式评价法作为判断依据。首版规则主要参考 Nielsen 启发式原则，例如系统状态可见性、一致性与标准、错误预防、用户控制与自由、帮助用户识别与恢复错误等。每条规则进一步拆分为可检查项，使系统输出更具可解释性。
- (3) 大语言模型作为语义理解与建议生成工具。模型并不直接替代规则，而是在规则框架内完成文本理解、问题解释与建议表达，从而平衡可控性与生成能力。
- (4) 结构化输出作为可演示结果。最终结果统一组织为“问题类别 - 命中规则 - 证据片段 - 风险说明 - 优化建议”的字段结构，便于汇报、讨论与后续导出。

4.5 核心模块设计

模块	主要职责	关键输入/输出	与其他模块关系
输入校验模块	检查文本是否为空、是否符合格式要求，并返回提示信息。	输入：UI 描述文本；输出：合法输入或错误提示	位于分析流程最前端，保证后续逻辑稳定
规则库管理模块	保存规则名称、检查点、触发条件和建议模板。	输入：规则配置；输出：可遍历规则集合	为检查执行模块提供判断依据
检查执行模块	遍历规则，对文本描述进行语义匹配与问题识别。	输入：文本 + 规则集合；输出：原始问题列表	是整个系统的核心分析引擎
结果结构化模块	将原始问题整理为统一字段，并补充风险与建议。	输入：问题列表；输出：结构化结果	直接面向课堂展示和文档输出

4.6 数据结构与输出格式示例

为了便于团队后续扩展，我将规则库和结果输出都设计为可配置的结构。下面给出首版字段示意。

```

rule_item = {
    "rule_id": "H-01",
    "rule_name": "系统状态可见性",
    "check_points": ["是否有清晰反馈", "操作后是否提示结果"],
    "risk_template": "用户可能无法确认当前状态，增加迷失与误操作风险",
    "advice_template": "增加状态反馈、按钮结果提示或过程性说明"
}

result_item = {
    "category": "反馈不足",
    "rule_name": "系统状态可见性",
    "evidence": "描述中未体现提交后的反馈信息",
    "risk": "...",
    "suggestion": "..."
}

```

4.7 核心代码示例

以下代码用于说明 Sprint 1 首版的核心实现思路：系统先遍历规则库，再依据关键词与语义条件形成问题列表，最后生成结构化结果。代码不强调工程化完整度，而强调可以支撑方案验证和课堂展示。

```

def run_heuristic_check(ui_text: str, rules: list[dict]) -> list[dict]:
    issues = []
    for rule in rules:
        evidence = f"描述中缺少与 {rule['rule_name']} 相关的信息"
        hit = any(point in ui_text for point in rule["check_points"])
        if not hit:
            issues.append({
                "category": infer_category(rule["rule_name"]),
                "rule_name": rule["rule_name"],
                "evidence": evidence,
                "risk": rule["risk_template"],
                "suggestion": rule["advice_template"]
            })
    return issues

def format_report(issues: list[dict]) -> str:
    blocks = []
    for idx, item in enumerate(issues, start=1):

```

```
blocks.append(  
    f'{idx}. [{item["category"]} {item["rule_name"]}\n"  
    f" 证据: {item["evidence"]}\n"  
    f" 风险: {item["risk"]}\n"  
    f" 建议: {item["suggestion"]}"  
)  
return "\n".join(blocks)
```

代码说明：run_heuristic_check() 负责完成规则遍历和问题识别；format_report() 用于将结果转换为便于演示与输出的文本形式。在后续 Sprint 中，可继续把简单的关键词匹配替换为更细致的语义判断，或者把结果输出扩展为表格与 JSON 双格式。

4.8 验收标准与 GitHub 说明

本次 Sprint 1 个人任务的验收标准包括：① 能提供一份结构清晰的启发式规则库；② 能对示例 UI 描述完成一轮自动分析；③ 输出结果中包含规则依据、风险说明与优化建议；④ 相关设计过程、技术原理与核心代码能够写入课程报告。

GitHub 链接说明：目前小组代码仓库计划由项目经理统一创建并整合成员内容。故本次个人文档先以设计说明和核心代码片段为主，待仓库建立后可将规则配置、示例代码与演示材料统一同步到团队仓库中。

五、Sprint 1 执行过程中的协作与冲突处理

敏捷开发强调团队自组织。本次 Sprint 1 中，我负责的技术方案工作与其他成员之间存在较强协作关系。例如，规则设计与竞品分析同学整理的规则材料，会直接影响我的规则库组织方式；PM 需要我提供足够稳定的技术表达，才能把它整合进最终汇报；资源与展示支持同学则需要基于我的流程设计制作更直观的示意内容。

在协作过程中，最可能出现的冲突并不是“谁做得多”，而是“做法是否一致”。例如，有成员可能更关注汇报展示效果，希望先做界面示意；而从技术方案角度看，若规则结构尚未稳定，过早做演示会导致后续频繁返工。对此，我的处理方式是：先通过会议明确 Sprint 1 的共同目标是完成技术闭环验证，再将“漂亮展示”放到次优先级；当任务发生竞争时，以是否直接支撑 MVP 为判断标准。

这种处理方式既符合敏捷中‘围绕价值排序’的思想，也能够降低项目在时间有限条件下的分散投入风险。

六、项目风险识别与风险矩阵

按照作业要求，报告需要识别技术风险、商业风险和项目风险，并对优先级最高的 10 项风险建立风险矩阵。这里采用“概率 × 影响”方式进行评估：概率分为 1-5，影响分为 1-5，综合等级按乘积划分为高、中、低。

序号	风险项	类别	概率	影响	等级	应对策略
R1	规则库定义不完整，导致检查结果覆盖不足	技术风险	4	5	高	优先完成核心规则集，采用可扩展字段结构
R2	模型输出不稳定，建议内容前后不一致	技术风险	4	4	高	采用模板约束与示例校准，减少自由生成
R3	Screen2Vec 描述文本质量不足，影响分析准确度	技术风险	3	5	高	先限定输入格式并准备标准示例
R4	Sprint 周期过短，任务无法全部完成	项目风险	4	5	高	严格限定 MVP 范围，优先保证主链路
R5	成员之间理解不一致，造成返工	项目风险	4	4	高	每日同步关键决策，统一术语和输出模板
R6	过度追求 Demo 展示，挤占规则设计时间	项目风险	3	4	中高	以 MVP 目标为先，Demo 仅做加分项
R7	课程项目缺乏真实用户验证，商业价值说服力不足	商业风险	3	3	中	以课堂场景和设计评审效率作为价值证明
R8	后续接入图片识别或复杂界面解析成本过高	技术风险	3	4	中高	首版坚持文本输入边界，后续再评估扩展
R9	团队仓库或文档管理不统一，影响合并效率	项目风险	3	3	中	统一命名规范、分支策略和提交节奏
R10	评审时无法清晰解释系统判	商业/项目风	2	5	中高	强化规则依据、证据

	断依据	险				片段和流程图表达
--	-----	---	--	--	--	----------

从以上 10 项风险看，优先级最高的风险主要集中在两个方面：一是技术基础是否稳定，如规则库是否完整、模型输出是否可控、输入文本质量是否足够；二是项目执行是否聚焦，如 Sprint 时间是否够用、团队是否对 MVP 有一致理解。这说明当前阶段最重要的不是盲目扩展功能，而是先稳定首版核心逻辑。

七、个人反思与下一步计划

通过本次 Sprint 1 规划，我更加明确了自己在团队中的价值并不是单纯“写代码”，而是把模糊的想法转化为可执行、可验证、可汇报的技术方案。技术负责人需要在‘做得深’和‘做得成’之间取得平衡：既要保证方案有一定技术含量，又要对课程项目的时间、范围和团队协作现实保持敏感。

下一步，我计划继续完善规则项与示例案例，推动团队建立统一仓库和命名规范，并在 Sprint 2 中考虑加入结果严重程度标记、异常输入提示或简单演示页面，使系统更接近一个可操作的课堂展示原型。

总体而言，本次作业已经完成以下核心目标：明确了 Sprint 1 的个人承诺、完成了软件模块设计报告、识别并分析了项目关键风险，也为后续迭代留下了清晰的扩展接口。